Nuska: Stage-Aware Communication Scheduling for Disaggregated LLM Serving

Yijun Sun[1] Xudong Liao[1] Songrun Xie[1] Hao Chen[2] Han Tian[3]
Wenxue Li[1] Yiming Zhang[2] Kai Chen[1]

[1]iSING Lab, Hong Kong University of Science and Technology
[2]Shanghai Jiao Tong University [3]University of Science and Technology of China

arXiv:2603.17456v2 [cs.NI] 19 Sep 2026

ABSTRACT

Meeting stringent Time-To-First-Token (TTFT) requirements is crucial for LLM applications. To improve efficiency, modern LLM serving systems adopt disaggregated architectures with diverse parallelisms, introducing complex multi-stage workflows involving reusable KV-block retrieval, collective communication, and P2D transfer. Flows from dependent stages overlap within and across requests on shared bottleneck links, making TTFT highly susceptible to network contention and necessitating stage-aware scheduling. Unfortunately, most existing works schedule flows in a stage-agnostic manner, leading to uncoordinated contention that constitutes a primary cause of SLO violations.

In this paper, we present Nuska, a holistic multi-stage flow scheduling mechanism designed to maximize TTFT SLO attainment. At its core, Nuska approximates the Least-Laxity-First (LLF) scheduling policy without requiring precise knowledge of a request's remaining slack. It achieves this through a *Defer-and-Promote* principle implemented through a Reverse Multi-Level Queue (RMLQ) structure. By dynamically promoting task precedence as effective laxity diminishes, Nuska prioritizes flows with less laxity while preventing requests with loose SLOs from prematurely consuming network bandwidth. We implement Nuska as a pluggable module integrated into vLLM, and evaluate it on a 8-server, 32-GPU testbed as well as through large-scale simulations. Our results demonstrate that Nuska effectively outperforms state-of-the-art baselines, improving the TTFT SLO attainment by 1.2×–2.4×.

1 INTRODUCTION

The rapid advancement of large language models (LLMs) has spawned a wide range of use cases, including interactive chatbots [20, 65, 101], code assistants [47, 105], and agentic systems [27, 81, 95]. Modern serving systems must sustain massive request volumes while adhering to heterogeneous service-level objectives (SLOs) [18, 29, 67, 76, 99] to meet stringent performance expectations. Among these metrics, time-to-first-token (TTFT) remains particularly critical, as it governs both the responsiveness required for interactive users [25, 87] and the strict timing constraints of agentic pipelines [37, 45, 57]. Violations of TTFT targets often trigger repetitive request retries [52, 53], leading to unnecessary resource consumption [70] and significant revenue loss [73].

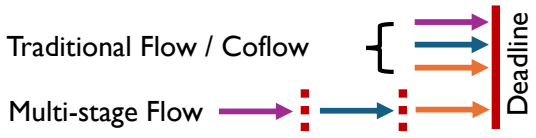


Figure 1: Illustration of difference between Multi-stage Flow Scheduling and prior stage-agnostic scheduling.

Modern LLM serving systems typically adopt disaggregated prefill–decode architectures with KV-cache reuse [26, 31, 32, 66, 69, 79, 93, 106] and leverage diverse forms of parallelism [9, 38, 46, 98] to improve efficiency. Despite their promise, they still suffer from substantial communication overhead during the prefill phase. Specifically, generating the first token involves *multi-stage* communication: (1) remote retrieval of reusable KV-blocks, (2) collective communication for the synchronization of intermediate activations, and (3) Prefill-to-Decode (P2D) transfer. Flows from different stages frequently overlap and contend for shared network fabrics, which leads to *intra-request contention*, where different communication phases within the same request interfere with each other, and *inter-request contention*, where communication from different batches or prefill units competes for shared links. Such contention significantly inflates prefill TTFT latency and heightens the risk of system-wide SLO violations.

Unfortunately, prior works fail to meet end-to-end deadlines under contention (Figure 1), largely because they treat flow in stage-agnostic manner (i.e., without awareness of their stage dependencies). Traditional flow scheduling schemes (e.g., Fair Sharing [7, 107], SJF [8, 10, 28, 41], EDF [28, 83, 88], etc.) operate on individual flows without holistic view. Even coflow-based approaches [4, 21, 22, 86, 97, 100] fall short:

while they manage data-parallel flows as a group, they ignore the inherent dependencies across stages. Consequently, these schemes may over-prioritize early-stage communication or starve latency-critical downstream stages, ultimately degrading TTFT SLO attainment.

To this end, we ask: *Can we schedule multi-stage communication holistically in LLM serving systems to maximize overall TTFT SLO attainment?*

When exploring the design space, we note distinct efforts that optimize isolated stages, targeting either collective communication [15, 16, 72, 77, 91] or KV-cache transfers [19, 51, 69, 79]. However, these specialized solutions remain insufficient. The reason is that they often pursue conflicting optimization objectives, naively combining them can inadvertently amplify network contention.

We answer this question affirmatively with Nuska, a *holistic* multi-stage communication scheduler that jointly orchestrates dependent communication stages to maximize TTFT SLO attainment. Our key insight is that the global TTFT deadline can be gradually translated into explicit flow-level deadlines as prefill execution progresses. Leveraging this insight, Nuska approximates the Least-Laxity-First (LLF) policy under uncertainty and realizes a *Defer-and-Promote* principle via a Reverse Multi-Level Queue (RMLQ). By dynamically promoting task precedence based on diminishing effective slack, Nuska prioritizes genuinely critical stages while preventing requests with loose SLOs from prematurely consuming network bandwidth.

Although our insight is straightforward, translating it into an efficient, real-world system requires addressing several non-trivial challenges. First, how to schedule last-stage communications with explicit deadline without incurring excessive prioritization? Second, how to schedule early-stage flows governed by implicit TTFT bound under uncertain laxity? Third, how to ensure Nuska's compatibility with emerging collective communication and KV-cache libraries?

For the first challenge, we employ a lazy promotion strategy leveraging Minimal Link Utilization (MLU) as the trigger. P2D flows initialize in low-priority queues and are promoted only when their MLU exceeds a predefined threshold. This ensures that P2D flows receive only the minimum bandwidth for just-in-time completion, preserving bandwidth headroom for early-stage communications. To avoid priority thrashing, promotions occur only at layer granularity. This approach enables effective coarse-grained prioritization while facilitating practical implementation on commodity switches with hardware priorities queues and lightweight packet tagging, thereby eliminating the risk of packet re-ordering.

To address the second issue, we utilize a two-tier scheduling strategy. For intra-request contention within a single prefill unit, we prioritize collective communication that unblock subsequent computation, while elevating KV-cache transfers only when they threaten to stall the next layer's execution. For inter-request contention across the prefill cluster, we order units based on TTFT deadlines and apply feasibility checks to prune infeasible communication tasks to avoid blocking. This design prevents premature promotion when laxity is unclear, while keeping early-stage communication aligned with downstream TTFT requirements.

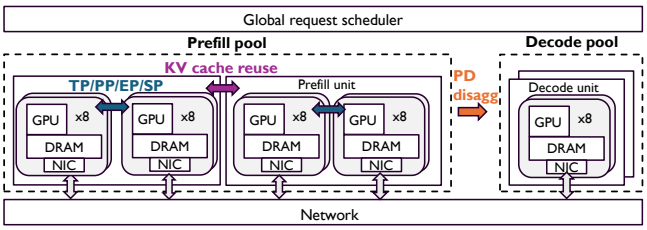


Figure 2: Illustration of LLM serving at scale.

Finally, to ensure compatibility, Nuska integrates with existing stacks (e.g., NCCL [60], Mooncake [36]) via lightweight task adapters. These adapters intercept tasks for precise host-side prioritization in software queues, while mapping priorities to DSCP values for robust traffic isolation on switches. By employing hybrid priority enforcement, Nuska ensures seamless integration and effective contention resolution using standard network primitives.

We build a Nuska prototype and evaluate it on a 8-server testbed with 32 NVIDIA 3090 GPUs [59], 16 Mellanox NICs [61], all connected to a single Top-of-Rack (ToR) switch. We implement Nuska as a pluggable module into NCCL and Mooncake, and integrate it with the vLLM inference engine. Using this prototype, we successfully demonstrated the benefits of Nuska on the state-of-the-art LLM [3].

To evaluate the performance of Nuska at scale, we further perform large-scale simulations using four representative real-world MoE models [1, 2, 23, 71]. Our results reveal that Nuska significantly outperforms state-of-the-art scheduling schemes, improving the TTFT SLO attainment rate by 1.2×-2.4× compared to baselines. We also observe that Nuska cutting down non-overlapped collective completion time by 50% and also improving the request earliness greatly.

2 BACKGROUND AND MOTIVATION

2.1 LLM Serving and TTFT

Large language models have become the foundation of modern AI services. These models are inherently autoregressive: the system first processes the entire prompt to generate the first token (the prefill stage), and then produces subsequent tokens iteratively using the accumulated KV cache (the decode stage). Performance is typically characterized by

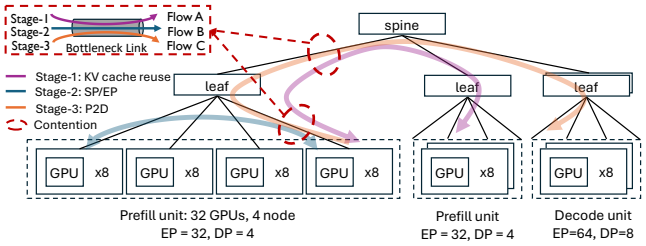


Figure 3: An example illustrating communication contention in LLM serving systems: KV cache transfers (Stage 1 & 3) and parallelism communication (Stage 2) compete for the same link bandwidth.

Time-to-First-Token (TTFT) for the prefill phase and Time-Between-Tokens (TBT) for the decode phase.

TTFT is a critical service-level objective (SLO) for many LLM applications. In interactive scenarios like chatbots and voice assistants, users expect an immediate initial response, and some studies indicate noticeable disengagement when latency exceeds a few hundred milliseconds and abandonment beyond seconds [35, 49, 84]. TTFT is also vital for automatic agentic frameworks, which rely on strict timeouts [37, 45] for fault tolerance. A delayed first token triggers costly remedial actions such as service retries [52, 53] or provider switching [57]. A recent report from Microsoft [73] reveals that the delayed first token is one of the primary source ($\sim$ 40%) of service failure (reported as timeout error), leading to revenue loss, higher support costs, and reputational risk.

As depicted in Fig. 2, modern production systems [46, 70] scale out to clusters comprising thousands of nodes interconnected with high-bandwidth links, where each node contains multiple xPUs (e.g., GPUs, TPUs, NPUs). Nodes are organized into individual serving units, each dedicated to host one model replica. Models are deployed using a combination of tensor [56], pipeline [55, 102], sequence [48, 89, 92], and expert parallelism [9, 38, 46, 98]. To further optimize efficiency, recent work [32, 66, 79, 106] proposes *prefill–decode disaggregation*, which assigns prefill tasks to compute-optimized devices and decode tasks to memory-rich devices, thereby accommodating their heterogeneous resource demands. In addition, *KV cache reuse* [26, 31, 69, 93] has been introduced to amortize prefill costs across requests by transferring cached contexts between nodes.

2.2 Contention Across Multi-Stage Communication

Generating the first token in existing production systems (Fig. 2) consists of multi-stage communications:

- **Stage 1: KV-cache reuse**: fetches reusable KV-cache blocks from remote prefill unit;
- **Stage 2: Collective communication**: exchanges intermediate activations across devices via model parallelism;
- **Stage 3: Prefill-to-Decode (P2D) transfer**: delivers the full KV-cache history to the decode unit [1].

Prior literature has highlighted the substantial overhead of individual stages, noting that collective communication (e.g, all-to-all, etc.) occupy 40–60% of end-to-end latency [39, 40, 92, 98] and KV-cache movement contributes a non-trivial share [19, 42, 51, 70, 94]. In this paper, we further identify a critical yet underexplored problem: collective communication and KV-cache movement frequently overlap in time and contend for shared network bandwidth. Fig. 3 illustrates the spatial location of contention: KV-cache transfers (Stage 1&3) and collective communication (Stage 2) traverse shared physical interconnects. This spatial co-location creates two primary forms of contention, as shown in Fig. 4:

- **Intra-request contention** occurs when communication flows interfere within a single request. Specifically, layer-wise KV-cache transfers (Stage 1&3) compete directly with the ongoing collective communication on other layers (Stage 2) for shared link bandwidth.
- **Inter-request contention** arises when concurrent requests from different prefill units interfere, resulting in frequent link competition in the switch links. Moreover, driven by skewed block popularity, multiple prefill units may converge on a single remote *victim unit* to fetch "hot" KV blocks, causing contention between the victim unit's local collective communication (Stage 2) and remote KV-cache fetching on its NIC bandwidth (Stages 1&3).

To quantify the performance degradation caused by such interference, we conduct measurements on a 16-GPU prefill cluster (50 Gbps/GPU) hosting two serving instances ($TP = 1, EP = 8$). We use QwenB-agent, an agent workload from production-derived Qwen traces [6], with average sequence length 1k tokens, 65% prompt reuse, and per-GPU request rate 1 req/s; the full testbed setup is described in Sec. 6.2. We evaluate the impact of contention on two metrics: end-to-end TTFT and collective communication time (CCT) of All-to-All operations (aggregating Dispatch and Combine phases). Specifically, we perform a comparative analysis between an ideal baseline (*w/o contention*) and a realistic serving scenario (*w/ contention*). Fig. 5a illustrates the TTFT distribution. The results reveal that, under contention, the overall TTFT is prolonged by nearly 50%. We attribute this inflation primarily to the latency degradation in All-to-All communication on the critical path, whose CCT nearly doubles (1.8×) due to contention with concurrent KV-cache operations (as shown

[1] Mainstream open-source serving frameworks [63, 74, 85] explicitly incorporate Stage 3 latency into the TTFT metric.

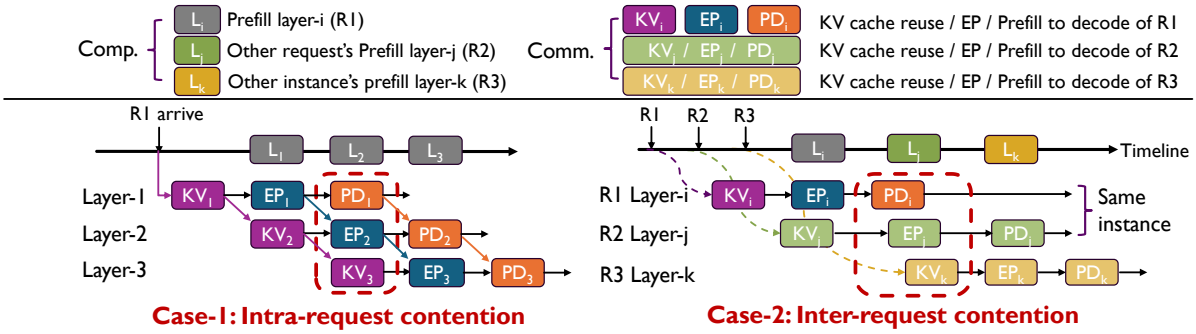
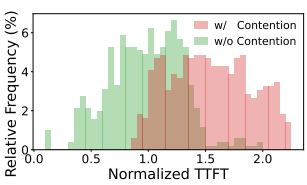
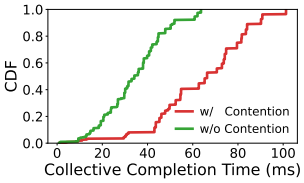


Figure 4: Illustration of two primary forms of contention in LLM serving systems.



(a) Impact of contention on end-to-end TTFT.



(b) Impact of contention on all-to-all latency.

Figure 5: [Testbed] Impact of communication contention on Mixtral 8x7B model.

in Fig.5b). This substantial slowdown confirms that network contention is a major source of performance variability in the prefill phase.

2.3 Limitation of Existing Work

The previous analysis identifies unmanaged multi-stage communication contention as a primary source of TTFT violations. We categorize existing approaches into two groups: recent system-level communication optimizations and conventional scheduling algorithms. As detailed below, they both fall short in coordinating such contention due to the lack of a holistic view of multi-stage dependencies.

Stage-agnostic flow scheduling. Conventional datacenter scheduling disciplines manage flows or coflows individually to optimize network metrics, such as flow/coflow completion time or deadline satisfaction. However, these objectives are misaligned with end-to-end application goals (e.g., TTFT attainment), because TTFT is jointly determined by flows across multiple interdependent stages. Consequently, these approaches inevitably over-prioritize slack-rich flows while starving the critical transfers required to unlock downstream computation, ultimately degrading TTFT SLO attainment.

Specialized optimization for isolated stages. We notice some distinct efforts optimize isolated stages, specifically targeting collective communication via algorithm synthesis [16, 34, 50, 75], parameter tuning [91], and overlapping [46, 98, 103, 104], or KV-cache transfers via pipelined prefetching [66, 69] and block coalescing [19, 42, 79]. However, these mechanisms often pursue conflicting objectives. Many achieve speedups by increasing communication concurrency to hide latency, yet none coordinates this concurrency across communication types. As a result, even with these optimizations, collective communication and KV-cache transfers often interfere with one another, which instead amplifies contention.

3 METHODOLOGY

3.1 Problem Formulation

Simplified Abstraction. Without loss of generality, we consider a single prefill request processing through L sequential Transformer layers. The communication workload for each layer can be generally abstracted as Multi-stage Flow (MsFlow). An MsFlow consists of three temporally dependent stages, where each stage constitutes a set of flow (or coflow) governed by the layer's lifecycle, which is detailed below:

- **Stage 1: Initialization.** Involves KV-cache reuse to transfer prerequisite states. This loosely coupled flow often overlaps with prior layer's computation to hide latency.
- **Stage 2: Execution.** Consists of collective communication (e.g., alltoall) that strictly blocks the subsequent computation.
- **Stage 3: Completion.** Handles the P2D transfer of results to decoding workers. This dictates the final token availability without blocking the current prefill computation.

For a single request, MsFlows collectively share one TTFT deadline and may partially overlap subject to layer dependency (e.g., Stage 2 of layer l must complete before computing layer $l+1$)

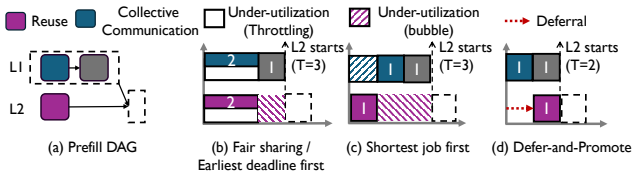


Figure 6: Comparison of scheduling policies for intra-request contention (ingress). (a) Ingress port contention during Layer-1 execution. (b)-(c) FS/SJF/EDF delays the start of Layer-2 ($T = 3$). (d) The *Defer-and-Promote* strategy advances the Layer-2 start time to $T = 2$ (-33%).

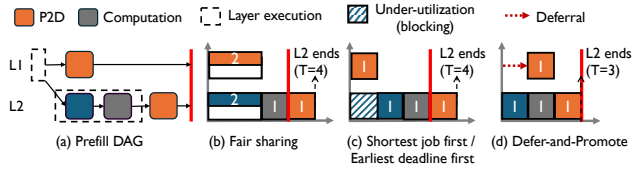
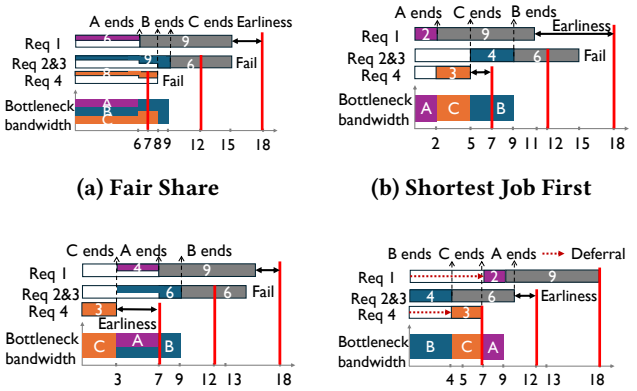


Figure 7: Comparison of scheduling policies for intra-request contention (egress). (a) Egress port contention during Layer-2 execution. (b)-(c) FS/SJF/EDF delays the end of Layer-2 ($T = 4$). (d) The *Defer-and-Promote* strategy reduces the Layer-2 finish time to $T = 3$ (-25%).

Table 1: An inter-request contention example.

Req ID	Flow ID	Flow Size	Remain time	Deadline
1	A	2	9	18
2,3	B	4	6	12
4	C	3	0	7



(a) Fair Share **(b) Shortest Job First**

(c) Earliest Deadline First **(d) Defer-and-Promote**

Table 2: Comparison of scheduling policies for inter-request contention. The red vertical lines denote hard deadlines, and gray bars are subsequent durations. (a) Fair Share and (b) SJF fail to protect the urgent Flow-B due to lack of deadline awareness, while (c) EDF completes flows with explicit but loose deadlines unnecessarily early. (d) *Defer-and-Promote* strategically defers non-urgent flows to minimize earliness, and promotes them only when necessary to achieve *just-in-time completion.*

Contention dynamics. Contention arises when concurrent MsFlows compete for shared network bandwidth. Recall Fig 4, intra-request contention occurs between MsFlows of different layers within the same request, while inter-request contention arises between MsFlows of distinct requests. Both cases simply reduce to contention between heterogeneous stages.

Overall Objective. The goal is to meet end-to-end TTFT deadline (SLO attainment) rather than optimizing individual flow latencies. Specifically, we must orchestrate the dependent MsFlows to ensure that all L MsFlows are completed within the deadline constraint. To achieve this, an ideal scheduler must balance two complementary objectives tailored to the specific semantics of MsFlow stages:

(i) **Minimizing Non-overlapped communication Latency.** For early stages (1&2), completing too late directly prolongs the current layer's execution time, postponing the release of the subsequent layer.

(ii) **Minimizing Earliness.** For the final Stage 3, completing too early yields no gain for meeting deadline but exacerbates potential network contention.

3.2 Key Observation and Scheduling Principle

Achieving optimality is theoretically intractable [11, 12]. To derive an effective heuristic, we first look into the execution dynamics of prefill. Our analysis reveals a structural property of request deadlines that motivates scheduling principle presented in this section.

Key observation. In the prefill stage, *request deadlines progressively materialize into flow-level deadlines as the prefill pipeline advances.*

Recalling Sec. 3.1, for the initial stages (1&2), the flow-level deadline is implicit, since it jointly determined by TTFT and duration of downstream tasks. As execution reaches the final stage, this constraint materializes: the global TTFT directly dictates the token return time, becoming an explicit flow-level bound. We borrow the concept of laxity (defined as the remaining budget before triggering deadline violation) from real-time systems to represent flow-level urgency. The distinct laxity reveals a scheduling opportunity: we can

safely defer last-stage flows (possessing deterministic laxity) to yield bandwidth for early-stage flows whose laxity is uncertain.

Scheduling principle: *Defer-and-Promote*. Leveraging the observation above, our key principle is to *defer non-urgent transfer until the latest safe moment, promoting priority only as necessary*, so as to minimize interference while ensuring timely completion. This principle translates into two concrete rules:

- **Principle #1 (Last stage flows with Explicit-deadline)** are initially deferred to yield bandwidth but gradually promoted to guarantee compliance.
- **Principle #2 (Early stage flows with implicit-deadline)** are deferred based on relative laxity and promoted as the laxity diminishes to mitigate the violation risks.

Why this works. By deferring non-urgent flows and selectively promoting them when necessary, the scheduler balances short-term efficiency with long-term deadline guarantees. This strategy improves TTFT SLO attainment along two axes: (i) minimizing the non-overlapping communication latency to shorten the prefill makespan, and (ii) regulating earliness to prioritize tighter requests under contention.

To see why *Defer-and-Promote* benefits the prefill DAG makespan, consider the consider the contention scenarios in Fig 6-(a) (ingress) and Fig 7-(a) (egress). Fair Sharing indiscriminately dilutes Stage 2 throughput; SJF (Fig. 6-(c), 7-(c)) preferentially prioritizes KV-movement flows (flow size typically smaller); and EDF over-prioritizes Stage 3 due to explicit deadlines (Fig. 6-(c)) and degenerates into Fair Sharing when deadlines are implicit (Fig. 7-(b)). Consequently, all three policies inflate the non-overlapped communication latency and stall computation. Defer-and-Promote resolves this by strategically deferring Stage 3 (higher laxity) to protect Stage 2, which demonstrating a significant reduction in makespan (Fig. 6-(d), 7-(d)).

Defer-and-Promote maximizes attainment under inter-request contention by strictly prioritizing truly urgent requests. Consider Table 1 as an example, where each request has an end-to-end TTFT constraint and implicit remaining time determined by downstream tasks. As illustrated in Fig. 2, three flows with varying sizes and deadlines contend for a bottleneck link. Unlike standard policies (e.g., FS, SJF, EDF) that often misjudge urgency, *Defer-and-Promote* prioritizes Collective Communication, as deferral would immediately stall the execution pipeline. It strategically defers other traffic: Stage 1 flows are promoted only when they threaten to stall the pipeline, while last-stage flows are delayed until their explicit deadlines.

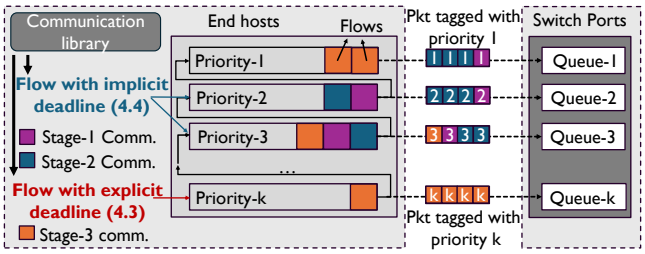


Figure 8: Design overview.

4 DESIGN

4.1 Design Challenges

While the *Defer-and-Promote* principle is conceptually simple, realizing it in LLM serving is challenging in practice. Nuska must make priority assignments *in the dark*, steering flows without precise laxity through a *discrete multi level queue* (e.g., finite switch queues). Specifically, we identify the following two challenges.

C-1: Scheduling flows with explicit deadlines without excessive prioritization. Even when flows carry explicit deadlines (e.g., P2D transfers), determining when to switch from defer to promote is non-trivial under discrete priority levels. Promoting too early wastes scarce bottleneck bandwidth, while promoting too late risks deadline misses. The challenge lies in striking the right balance for just-in-time completion given finite priority classes.

C-2: Scheduling implicit-deadline flows without precise laxity. The absence of precise laxity traps the scheduler in an error-prone dilemma. On one hand, underestimating laxity incurs unintended execution stalls, as mistakenly deferring Stage 2 flows immediately idles the GPU and inflates request latency. On the other hand, overestimating laxity results in priority inversion, squandering scarce bandwidth on deferrable work and starving truly urgent requests.

4.2 Overview

Figure 8 illustrates Nuska's core abstraction: the *Reverse Multi-Level Queue (RMLQ)*. To realize the *Defer-and-Promote* principle, RMLQ inverts the classic MLFQ paradigm (which demotes flows over time); instead, it initializes flows in low-priority queues to enforce deferral, promoting them strictly when diminishing laxity necessitates immediate execution.

At a high level, Nuska schedules all multi-stage communication through a shared RMLQ substrate, while applying separate rules (initial priority and promotion pace) for different stages. For last stage flows with explicit deadlines, Nuska employs Minimal Link Utilization (MLU), promoting a flow only when the remaining bandwidth is just insufficient to meet its deadline, thereby avoiding premature over-prioritization.

For early stage flows with implicit deadline, Nuska speculate relative urgency via Relative Layer Index (RLI). Flows begin at lower tiers climb up when it threaten to stall computation. Finally, to resolve potential cross-stage contention, Nuska reserves the highest priority level for deadline-explicit flows to avoid violations, and prioritizes deadline-implicit flows in the remaining levels to opportunistically exploit available bandwidth.

The rest of this section is organized as follows. §4.3 introduces scheduling flows with explicit deadlines. §4.4 describes the strategy for implicit deadlines. Finally, §4.5 combines these components to present the complete RMLQ arbitration logic.

4.3 Scheduling with Explicit Deadline

To address C-1, we need to ensure the last-stage communication task is judiciously deferred—neither excessively prioritized nor starved—while strictly guaranteeing the request deadline. The challenge lies in mapping continuous urgency levels derived from these deadlines onto K discrete priority queues. Consequently, the scheduler needs a mechanism to approximate continuous deadline-derived urgency using coarse-grained priority levels, which requires determining how to quantify urgency, when to trigger promotion, and at what granularity to apply scheduling decisions (e.g., per-packet or per-layer).

Specifically, consider an L-layer model serving a batch of request. At each layer ℓ, the P2D stage emits a set of flows. Flows within the same request r inherit its TTFT deadline D_r, while flows from different requests are scheduled independently. We assume there are K priority queues $P_i (1 \le i \le K)$ where P_1 has the highest priority. We denote the threshold for promoting the priority from P_j to P_{j-1} as τ_j $(2 \le j \le K)$. we define $\tau_K = +\infty$ ensuring that even the extreme loose flows are captured in the lowest-priority queue.

Urgency metric. We define *Minimal Link Utilization (MLU)*: $\mathrm{MLU}_i(t) = \frac{Size_{rem}(t)}{Time_{rem}(t) \cdot B \cdot (1-\rho)}$, where $B(1-\rho)$ represents the effective bandwidth after accounting traffic load ρ. This metric represents the minimal share of the residual link capacity required to satisfy the deadline. Crucially, $\mathrm{MLU}_i(t) > 1$ signifies an infeasible overload state; values approaching 1 signal critical urgency requiring exclusive service, while lower values imply sufficient slack for deferral. By contrast, purely deadline-based metrics (e.g., EDF) are ill-suited since they suffer from the domino effect under overload [13] and further degrade when constrained to discrete priority levels [14].

Optimizing promotion threshold. Deriving the globally optimal thresholds that minimize deadline miss rates is computationally intractable (NP-hard). Therefore, we seek a robust, practical approximation by minimizing the relative quantization error introduced when mapping continuous urgency to discrete levels. When a flow with urgency $v \in [U_{min}, U_{max}]$ is mapped to a discrete priority level τ_k, it incurs a relative error of $\epsilon \approx \frac{|v-\tau_k|}{v}$. Our intuition here is to minimize the worst-case relative error across all threshold. $\prod_{k=1}^{K} r_k = \frac{u_2}{u_1} \cdot \frac{u_3}{u_2} \cdot \cdots \cdot \frac{u_K}{u_{K-1}} = \frac{u_K}{u_1} = \frac{U_{\max}}{U_{\min}}$. Mathematically, this minimum is achieved if and only if all ratios are equal ($r_k \equiv r$). This necessitates a geometric progression for threshold generation (where $r = (U_{\max}/U_{\min})^{\frac{1}{K-1}}$). Since the precise bounds $U_{\max}$ and $U_{\min}$ are often unknown or dynamic in practice, we approximate this geometric spacing by configuring the promotion threshold as $Q_i = E^{-i} \cdot U$ $(1 \le i \le K-1)$, where parameters E and U are set empirically (e.g., $E = 4, U = 0.5$) to yield robust performance.

Promotion behavior. Fine-grained promotion decision may adversely introduce practical concerns in network scheduling. In particular, if promotion is applied at a fine granularity (e.g., per packet), a single message may be fragmented across multiple priority queues. This can lead to packet reordering, where later packets overtake earlier ones, triggering costly transport-layer recovery and reducing effective throughput. To avoid this issue, we restrict promotion to layer boundaries, ensuring priority atomicity at the message level. Crucially, this coarser granularity does not compromise scheduling expressiveness, as the substantial depth of modern LLMs ($L > K$, e.g, L=64 and K=8/16) provides ample room for priority adjustments. Additionally, we enforce a monotonic promotion policy (i.e., only allowing promotion). This prevents priority oscillation and reinforces the *Defer-and-Promote* principle, ensuring that flows are conservatively maintained in lower tiers until urgency strictly necessitates promotion.

4.4 Scheduling with Implicit Deadlines

To address the lack of precise laxity under implicit deadlines (Challenge C-2), Nuska exploits structural determinism to infer relative urgency within requests, and resolves residual contention across requests using robust arbitration mechanisms.

4.4.1 Intra-request Scheduling with Structural Determinism. While precise laxity is elusive due to dynamic scheduling shifts, the relative laxity within request is deterministic: flows blocking immediate execution strictly more urgent than lookahead transfers that enable future progress.

We quantify this using the *Relative Layer Index*, defined as $\mathrm{RLI} = L_{\text{target}} - L_{\text{curr}}$, where L_{curr} is the index of the currently executing layer and L_{target} is the target layer index where the data is consumed. A smaller RLI indicates a tighter safe

deferral window. Nuska resolves contention by strictly prioritizing flows with the smallest RLI. We apply it to resolve two representative contention cases detailed below.

Case I: Collective vs. KV Reuse. When collective^i competes with reuse^j ($j > i$), the collective targets the current layer ($L_{\text{target}} = i$), yielding RLI(coll) = 0. In contrast, the reuse targets layer j with RLI(reuse) > 0. We therefore enforce collective$^i \succ$ reuse^j to avoid execution stalls.

Case II: Inter-KV Reuse Contention. Under multi-layer lookahead, flows reuse^j and reuse^k ($L_{\text{curr}} < j < k$) often contend. Comparing their urgency, layer j represents a nearer deadline, implying RLI(j) < RLI(k). Prioritizing the smaller RLI effectively delays the onset of the earliest execution stall. Thus, we enforce reuse$^j \succ$ reuse^k.

In summary, Nuska unifies these decisions by strictly prioritizing flows with the smallest RLI. Theorem 1 formalizes the optimality of this structural ordering.

Theorem 1. *Assuming an ideal computation model without preemption overhead, the prefill makespan is minimized by a schedule that strictly prioritizes flows with the smallest RLI.*

Proof sketch. Intuitively, prioritizing a high-RLI flow during a stall wastes the opportunity to overlap it with subsequent computation. We prove this via an exchange argument: shifting any flow with a larger RLI out of a stall interval into a later overlapped region strictly reduces the current blocking duration without violating future dependencies. We provide the detailed proof in Appendix.

4.4.2 Robust Inter-request Scheduling. With intra-request contention resolved by RLI (§4.4.1), the remaining challenge is breaking ties *across requests* when competing flows have similar RLI and simultaneously threaten execution. The scheduler must protect requests with high SLO violation risks under imprecise laxity. However, synchronous batch execution exacerbates the impact of scheduling pitfalls, allowing errors to propagate to all batched peers. Our intuition is not to recover exact laxity, but to make scheduling decisions *robust to estimation errors*, prioritizing requests/batches where delays would most severely hurt SLO attainment.

Scheduling pitfalls under imprecise laxity. We identify two representative failure cases arising from misjudging laxity:

- **The Piggyback effect.** In batches with mixed deadlines, a small number of extremely tight requests can dominate the inferred batch-level urgency. Consequently, loose requests in the same batch inherit elevated priority, preempting other batches that are globally more urgent.
- **The Black Hole effect.** Proximity to a deadline does not guarantee feasibility. Under overload, a batch may appear urgent while carrying a workload that exceeds available bandwidth. Prioritizing such batches wastes resources on inevitable failures, blocking other viable batches.

We define the *Robust Effective Deadline* (*RED*) to quantify urgency while counteracting the Piggyback effect. We partition a batch into tight and loose sub-batches at the maximal deadline gap. With $D^T_{\min}$ and $D^{Lo}_{\min}$ representing the minimum deadline of each sub-batch, we define $RED = f \cdot D^T_{\min} + (1-f) \cdot D^{Lo}_{\min}$, where f is the proportion of tight requests. When tight requests are rare (small f), *RED* naturally shifts towards the loose deadline ($D^{Lo}_{\min}$), preventing outliers from hijacking the batch priority.

Complementing RED-based ordering, we employ overload control to mitigate the Black Hole effect. The intuition is to prevent doomed or excessively heavy requests from blocking viable ones. We perform a worst-case feasibility check by accumulating estimated computation and communication delays. We iteratively prune the requests contributing the most to the cumulative delay to restore feasibility for the remainder of the batch.

The full algorithm, detailed in the Appendix, synthesizes these strategies. Triggered upon batch arrival and departure, it outputs two key results: (1) an ordered sequence σ derived from RED, and (2) a set of pruned requests $\mathcal{H}$ identified as infeasible. We deliberately avoid fine-grained per-layer updates to prevent the scheduler from over-reacting to transient load estimation jitter.

4.5 Putting Everything Together

Nuska schedules all traffic through a unified RMLQ substrate. We classify flows according to stage feature and apply distinct rules for initial priority assignment and subsequent promotion.

- For last stage flows with explicit deadline, we calculate the MLU defined in§4.3 to determine the initial priority level. For promotion, Nuska updates priority at layer boundaries while the request is computing, and switches to periodic updates at fixed intervals after computation finishes.
- For early stage flows with implicit deadline, priority assignment relies on the RLI defined in§4.4.1. Stage 1 flows are initialized based on RLI and promoted incrementally at layer boundaries to align with computation progress. Stage 2 flows directly enter the high priority queue.

Arbitration. Nuska strictly reserves the highest priority for the most urgent last-stage flows. Within the remaining priority levels, Nuska adopts a hierarchical policy: it grants precedence to early-stage flows over last-stage flows to opportunistically exploit available bandwidth; to break ties among early-stage flows sharing the same RLI, Nuska adheres to the rank determined by the inter-request scheduling (§4.4.2).

5 IMPLEMENTATION

We implement Nuska in approximately 10,000 lines of code. Nuska employ two-tier control plane composed of local daemons and a centralized coordinator. Local daemons maintain scheduling context and enforce scheduling decisions, while reporting request-level statistics to the coordinator upon scheduling and completion events. The coordinator operates at request granularity, aggregating these summaries to perform inter-node tie-breaking, and disseminates the resulting decisions back to local daemons for enforcement.

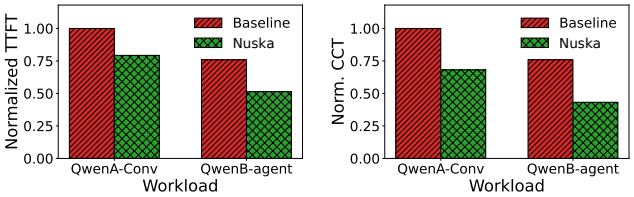
System Integration and Compatibility. We implement Nuska as a pluggable module that integrates transparently with NCCL and Mooncake. Each node runs a lightweight local daemon that interfaces with runtimes via lock-free MPSC queues over shared memory, enabling low-latency intra-node signaling. By intercepting low-level network primitives (e.g., RDMA Verbs), Nuska redirects work requests from the library's proxy threads to the local daemon, which orchestrates task execution according to our scheduling logic. The interaction between the runtime and the scheduler is governed by three standardized primitives: (1) `submit` registers tasks with metadata, (2) `permit` gates transmission by granting specific priorities, and (3) `completion` updates the internal state upon hardware signals. Requests can be pruned by modifying their metadata to suppress communication.

Priority enforcement. Commodity hardware typically exposes a limited number of priority classes ($K \ll L$), making direct mapping infeasible. In practice, collective traffic is typically confined to a regional serving unit, whereas KV-cache traffic spans the network across racks. We employ a hybrid priority queue tailored for such traffic. Specifically, Nuska enforces strict priority at the host via software queues (arbitration logic in §4.5), while leveraging DSCP-based hardware priorities to isolate KV-cache traffic within the network fabric. For flows with explicit deadlines, we calculate MLU threshold derived from K and the current network load ρ, and assign DSCP tags according to this threshold. For flows with implicit deadlines, we map their RLI to the physical queue range $[0, K-1]$ by capping the RLI value at $K-1$.

6 EVALUATION

6.1 Experiments setup

Testbed setup. The testbed consists of 8 servers, each equipped with 4 NVIDIA GeForce RTX 3090 GPUs, 40 CPU cores (Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz), 128GB RAM, and two ConnectX-5 100Gbps NIC. The servers are connected via NVIDIA SN2700 Ethernet switches. Within each server, the GPUs communicate via PCIe Gen 3.0. All servers run Ubuntu 22.04 with CUDA 12.8, PyTorch 2.8.0, and NCCL 2.28. we configure a prefill cluster comprising 16 GPUs, while decode workers are provisioned separately. For simplicity, we limit the decode output length to one token, since it does not affect TTFT.



(a) Normalized TTFT. **(b) Normalized CCT.**

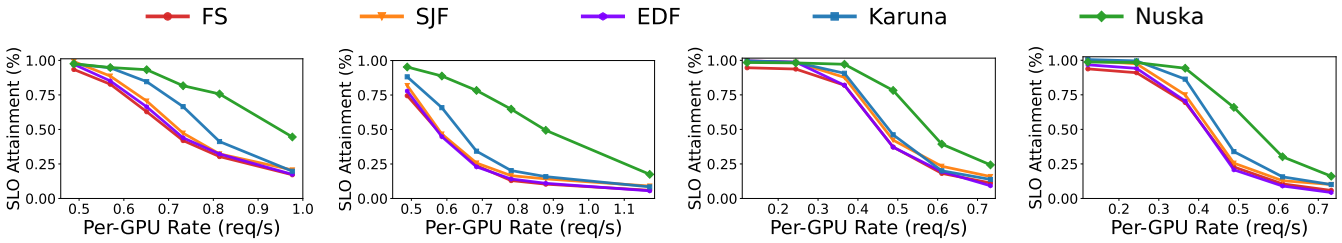
Figure 9: [Testbed] Mixtral-8x7B

Simulation setup. Our simulator is built upon Vidur [5] and the flow-level simulator flowsim [82]. We extend Vidur to support the Mixtral model with expert and sequence parallelism, as well as PD disaggregation with KV-cache reuse. The simulation workflow begins with offline profiling of operator latencies via Vidur profiler. At runtime, we employ a unified event-driven mechanism: Vidur simulates the LLM serving behavior to generate computation and communication tasks, while flowsim parser the communication task and trigger the underlying network events. Crucially, both computation events and network events are processed within a single event queue to ensure correctness. By default, we simulate a 256-server cluster with a 1:1 prefill-decode worker ratio, managed by a KV-cache aware scheduler similar to Dynamo [62]. Each server hosts eight GPUs interconnected via NVSwitch (900 GB/s) and eight NICs, connected via a 1:1 fat-tree network with link bandwidth 200Gbps. The latency profiles are calibrated based on NVIDIA A100 GPU.

Metrics. We primarily evaluate Time-to-First-Token (TTFT) and its Service Level Objective (SLO) attainment rate. Following the methodology in [29, 67, 78], we define the SLO threshold as 3× the TTFT measured under low-load conditions by default. We also evaluate microbenchmarks such as the collective completion time (CCT) and request earliness to help interpret the observed performance gains.

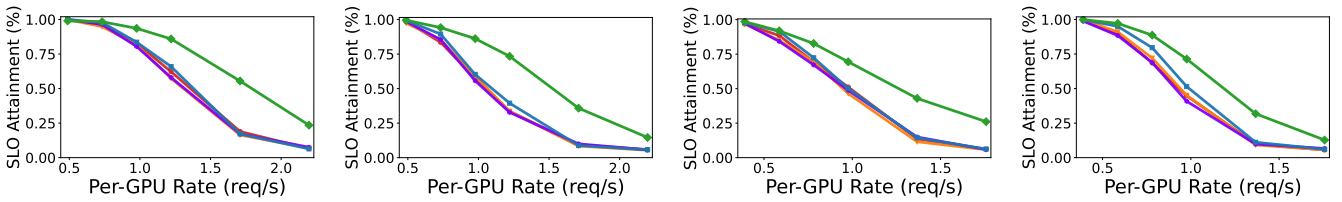
6.2 Testbed Experiments

Model and workload. In the testbed experiment, We evaluate the Mistral 8x7B [3] MoE model (FP16, top-k=2), configured with a tensor parallelism (TP) degree of 1 and expert parallelism (EP) of 8. For the workload, we utilize two production traces derived from Qwen [6]. QwenA-Conv represents a *conversation workload* with an average sequence length of 2k tokens and a 50% prompt reuse rate. QwenB-agent represents an *agent workload*, characterized by an average length of 1k tokens and a higher reuse rate of 65%. Request arrivals follow a Poisson process, where we vary the requests



(a) Mixtral-8x22B (b) DBRX (c) Qwen3-Coder (d) Grok2

Figure 10: [Simulation] Performance on conversation workload (QwenA)



(a) Mixtral-8x22B (b) DBRX (c) Qwen3-Coder (d) Grok2

Figure 11: [Simulation] Performance on agent workload (QwenB)

per second (RPS) to modulate system load. Due to testbed capacity limits, we clip the first 512 requests of each trace as a warm-up phase and use the subsequent 1024 requests for evaluation.

Figure 9 reports how Nuska on QwenA-Conv and QwenB-Agent workload. We configure vLLM with 8192 batched tokens and set the per-GPU request rate to one request per second. Nuska significantly reduces TTFT across both workloads, optimizing the mean TTFT by 20.7% (1.26×) on QwenA-Conv and 32.3% (1.48×) on QwenB-Agent. The TTFT reduction is primarily driven by faster completion of computation-blocking collectives. As shown by the all-to-all completion time (CCT), Nuska shortens stage-2 communication by 31.9% (1.47×) on QwenA-Conv and 43.1% (1.76×) on QwenB-Agent. These collectives lie directly on the critical path of prefill execution: delaying them immediately stalls GPU computation and inflates end-to-end latency.

6.3 Large-Scale Simulations

In the large-scale simulations, we expand the evaluation scope to cover a diverse spectrum of production-grade architectures and workloads.

- We examine a comprehensive set of mainstream MoE models with expert parallelism, ranging from architectures with large but few experts (e.g., Mixtral 8x22B [2] and Grok-2 [1], configured with TP=4, EP=8) to models with small but many experts (e.g., DBRX [23] with TP=2, EP=16, and Qwen3-Coder [71] with TP=1, EP=32). These models are deployed with 32-GPU instance utilizing a hybrid parallelism strategy. We evaluate these MoE models using Qwen conversation and agent traces to represent typical interactive behaviors.
- We further assess the effectiveness of Nuska on dense models with sequence parallelism [48]. We evaluate the Llama [54] model configured with a 1M token context window. In this setup, the model is deployed on 16-GPU instance using TP=4 and SP=4. We pair this model with the Mooncake dataset [70] to simulate real-world conversation and agent workloads characterized by similar reuse ratios ($\sim$ 40%, $\sim$ 65%) and extended context lengths ($\sim 15k$, $\sim 9k$).

Baselines. We compare Nuska against four classic flow scheduling policies:

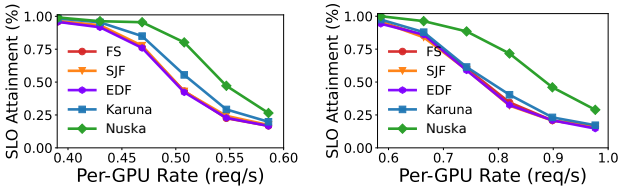
- **Fair Sharing** enforces max-min fairness among concurrent flows by allocating bandwidth equally, regardless of flow sizes or deadlines.
- **Shortest Job First (SJF)** minimizes average flow completion time by strictly prioritizing flows with smaller flow sizes.
- **Earliest Deadline First (EDF)** prioritizes flows with explicit deadlines. Since application-level deadlines do not directly translate to individual network flow deadlines, it applies fair sharing for flows with implicit deadlines.
- **Karuna [17]** allocates the minimum required bandwidth to flows with deadlines to ensure on-time completion, while scheduling the remaining flows using SJF.

End-to-End TTFT SLO Attainment. We evaluate the end-to-end TTFT SLO attainment under varying request rates across diverse models and workloads. As shown in Fig 10, 11, and 12, Nuska consistently outperforms state-of-the-art baselines across all scenarios.

Figure 10 details the performance of Nuska on conversation workloads for diverse MoE models. Specifically, under high request rates, Nuska achieves 1.4×–1.8× higher SLO attainment for Mixtral (Fig. 10a), Qwen Coder (Fig. 10c) and Grok (Fig. 10d), and up to 2.4× for DBRX (Fig. 10b). Furthermore, Nuska can sustain 1.17×–1.46× higher request rates than the strongest baseline (Karuna) while maintaining the similar SLO attainment. In conversation workload, a small fraction of tail requests necessitates large KV-cache movements, which precipitate contention across Stage 1 to Stage 3 under high load. Nuska outperform baselines significantly since it successfully coordinates multi-stage contention. Among the baselines, Karuna performs relatively better because it *happens to* defer non-urgent Stage 3 traffic and provides partial relief; however, without explicit multi-stage coordination, it eventually succumbs to contention as other baselines.

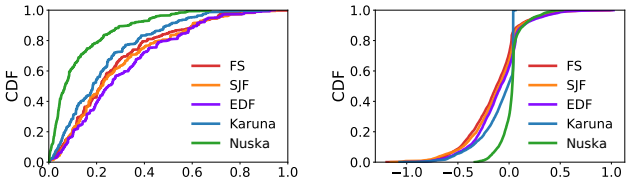
Figure 11 details the performance of Nuska on agent workloads for diverse MoE models. Focusing on the mid-to-high load regime, Nuska achieves 1.4×–1.7× higher SLO attainment for Grok-2 and Mixtral, and reaches 2.0× for DBRX and Qwen-Coder. Furthermore, Nuska sustains 1.2× higher request rates for Grok-2, ~ 1.3× for Mixtral and Qwen-Coder, and up to 1.4× for DBRX compared to the Karuna. Compared to conversation traces, agent requests have shorter prompt length but exhibit higher average reuse rates, with multiple concurrent requests sharing identical prefixes. This pattern induces severe one-to-many sender-side contention, which primarily concentrates on Stage 1 and Stage 2 as multiple workers simultaneously contend for the same cached states. Nuska effectively mitigates this contention by protecting urgent flows without requiring precise laxity. Notably, the performance gap between Karuna and other baselines narrows in this scenario. When contention is not dominated by Stage 3 traffic, Karuna falls back to SJF (deadline is implicit) to schedule Stage 1 and Stage 2 flows; however, being unaware of stage semantics, this approach still exposes tight requests to SLO violations as other baselines.

Figure 12 details the performance of Nuska on two representative long-context workloads using the Llama3-8B model with Sequence Parallelism (SP). For the Mooncake dataset, the conversation and agent workloads follow request patterns similar to those in the Qwen experiments (e.g., reuse ratios, access patterns), but feature significantly longer context lengths. In the conversation workload (Fig. 12a), Nuska demonstrates robust performance under increasing load, achieving 1.3×–1.6× higher SLO attainment against



(a) Conversation workload **(b) Agent workload**

Figure 12: [simulation] llama3-8B (Mooncake dataset)



(a) Normalized CCT. **(b) Normalized earliness.**

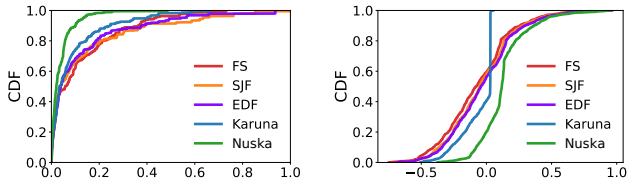
Figure 13: [Simulation] Breakdown on DBRX (Qwen-A, Rate=0.7)

Karuna. This advantage is further amplified in the agent workload(Fig. 12b), where Nuska attains 1.4×–1.9× higher SLO attainment and supports up to 1.15× higher per-gpu request rates given tight SLO budget.

Breaking down the performance gains. We further conduct micro-benchmarks to analyze the source of the gains. As shown in Figure 13 and Figure 14, Nuska successfully (i) minimizes Collective Completion Time (the non-overlapping communication latency) to shorten the prefill makespan, and (ii) regulates earliness to protect truly urgent requests under contention.

Figure 13 breaks down the performance gains that lead to the 2.2× higher SLO attainment achieved by Nuska on the DBRX model. The first source of improvement comes from reduced execution time during the prefill phase. Specifically, Nuska lowers the average CCT of expert parallel by 52% (Figure 13a). This reduction is mainly attributed to our stage-aware policy, which protects flows that block computation while deferring non-critical traffic to exploit available slack for overlap. As a result, Nuska significantly shortens the prefill makespan by lowering the effective communication overhead.

Beyond latency reduction, Nuska further improves SLO attainment by better prioritizing truly urgent requests under contention. We characterize this effect using *earliness* of request, where large positive values indicate unnecessary early completion that can block urgent requests, while negative values correspond to deadline misses. As shown in Figure 13b, Karuna exhibits the smallest non-negative earliness, but at the cost of a high violation risk, as it conservatively



(a) Normalized CCT. (b) Normalized earliness

Figure 14: [Simulation] Breakdown on Llama3 (Mooncake-Conv, Rate=0.5)

allocates near-minimal rates and fails to exploit available bandwidth. In contrast, other baselines show much larger non-negative earliness, indicating that they schedule flows without effectively prioritizing urgency. Nuska reduces the non-negative portion of earliness by 42% comparing with FS, SJF, EDF. By deferring Stage 3 flows to complete *just-in-time* and regulating early-stage traffic with a RED-based ording, Nuska preserves resources for turely urgent requests. Nuska adopts a priority-based scheduling that can exploit available bandwidth when no higher-priority flows are pending.

We observe a similar trend in Figure 14. Across both micro-benchmarks, Nuska consistently achieves higher SLO attainment by simultaneously reducing the collective completion time of sequence parallel (Figure 14a) and regulating earliness under contention (Figure 14b). These results indicate that the performance gains of Nuska are robust across models and are not tied to a specific workload or architecture.

7 DISCUSSION

Additional parallelism. While Nuska primarily focuses on Expert Parallelism and Sequence Parallelism, we acknowledge other strategies not explicitly covered in our evaluation. Tensor Parallelism, widely adopted in serving, is typically confined to intra-node fabrics (e.g., NVLink) without directly contending for inter-node bandwidth. Similarly, we do not explicitly discuss Pipeline Parallelism, as the prefill stage is usually latency-sensitive (requiring low or zero PP degrees [106]), and its communication volume is orders of magnitude smaller than KV cache movement or collective communications [44]. Should these flows appear on the shared network, they would be identified as Stage 2 traffic and remain compatible with our analysis. This observation extends to emerging parallelism strategies, which may introduce novel communication patterns with enhanced overlapping capabilities, yet still block dependent computation if delayed.

Hybrid model deployment. In practice, service providers often deploy a tiered model portfolio, consisting of a single large flagship model alongside multiple smaller models serving diverse workloads [65]. In such settings, network contention primarily arises from concurrent KV cache transfers (Stage 1 and Stage 3) across different models, as collective communication is typically confined within isolated racks or dedicated pods [46]. While Nuska is not explicitly evaluated under multi-model deployment, its scheduling principle remains applicable to KV cache traffic on shared networks. While promising, the holistic orchestration of such multi-model contention landscapes remains an under-explored design space.

Applicability to scale-up fabrics. While Nuska focuses on scale-out networks, its design principles could be extended to scale-up domains (e.g., NVL [58], UB [108],etc.). Contention between collective and KV Cache movement remains when multiple serving instance deployed together. Nuska's scheduling policy can be effectively adapted to such architectures by leveraging hardware-level isolation mechanisms (e.g., virtual channels or traffic classes), which could be exploited to enable differentiated prioritization.

Compatibility with GPU-initiated collective communication. While emerging GPU-initiated communication [24, 64] optimizes small-message latency for the decoding phase, it remains uncommon in prefill deployments due to strict hardware prerequisites and marginal gains for large-volume transfers. Nevertheless, even in such environments, Nuska ensures compatibility via CPU-assisted variants (e.g., host-side doorbells), while this method may introduce negligible latency overhead compared to a fully GPU-initiated execution [68].

8 RELATED WORK.

Flow and co-flow scheduling. Traditional flow scheduling optimizes per-flow metrics such as average flow completion time [7, 8, 10, 28, 41, 43, 107] or deadline satisfaction [28, 83, 88, 96] using policies like SJF and EDF. Coflow scheduling [4, 21, 22, 80, 86, 97, 100] extends this abstraction by grouping related flows and optimizing Coflow Completion Time for data-parallel workloads, while mixed-flow scheduling [17] considers the coexistence of deadline-sensitive and best-effort traffic. In contrast, Nuska explicitly accounts for multi-stage dependencies to schedule flows toward end-to-end TTFT attainments.

Optimization for individual stages. Prior research focuses on optimizing specific communication phases in isolation. For collective communication, efforts improve performance by algorithms synthesis [16, 75], resolving potential contention [15, 72], or enhancing overlap through system-level pipelining [30, 98, 103],. In parallel, other works optimize KV cache transfers by hiding latency via prefetching [66, 69], improving efficiency via block coalescing [19, 42] and multi-path transmission [90], or reducing data volume by trading off model quality [51]. However, these approaches

treat each stage in isolation, overlooking the potential contention between collective communication and KV cache transfers.

LLM serving systems. Recent advancements in LLM serving have evolved along two complementary dimensions. Architecturally, systems adopt prefill-decode disaggregation [66, 106] and KV-cache reuse [26, 69] to improve throughput and avoid redundant computation. On the orchestration front, numerous request schedulers [18, 29, 33, 78, 99] have been proposed to maximize SLO attainment. However, these systems primarily focus on optimizing compute efficiency under ideal network assumptions. Nuska is orthogonal to these approaches. It bridges the gap by addressing the network contention that they overlook.

9 CONCLUSION

This paper presented Nuska, a holistic multi-stage flow scheduling framework for LLM serving that targets end-to-end TTFT SLO attainment. Nuska is built on the observation that request-level deadlines progressively materialize into explicit flow level deadline as prefill execution. At its core, Nuska realizes a Defer-and-Promote scheduling principle that approximates Least-Laxity-First behavior without requiring precise laxity estimation. Through a prototype implementation and extensive evaluation, we show that Nuska consistently improves TTFT SLO attainment across diverse models and workloads.

REFERENCES

[1] [n. d.]. Grok. https://x.ai/news/grok-2. ([n. d.]).

[2] [n. d.]. Mixtral 8x22B. https://huggingface.co/mistralai/Mixtral-8x22B-Instruct-v0.1. ([n. d.]).

[3] [n. d.]. Mixtral-8x7B-Instruct-v0.1. https://huggingface.co/mistralai/Mixtral-8x7B-Instruct-v0.1. ([n. d.]).

[4] Saksham Agarwal, Shijin Rajakrishnan, Akshay Narayan, Rachit Agarwal, David Shmoys, and Amin Vahdat. 2018. Sincronia: Near-optimal network design for coflows. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 16–29.

[5] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.

[6] Alibaba-Edu. 2025. Qwen-Bailian Anonymous Dataset: Production-derived LLM Usage Traces. https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon. (2025). Accessed: 2026-02-06.

[7] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*. 63–74.

[8] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: Minimal near-optimal datacenter transport. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 435–446.

[9] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.

[10] Wei Bai, Li Chen, Kai Chen, Dongsu Han, Chen Tian, and Hao Wang. 2015. {Information-Agnostic} flow scheduling for commodity data centers. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 455–468.

[11] Riccardo Bettati and Jane WS Liu. 1990. Algorithms for end-to-end scheduling to meet deadlines. In *Proceedings of the Second IEEE Symposium on Parallel and Distributed Processing 1990*. IEEE, 62–67.

[12] Riccardo Bettati and Jane W-S Liu. 1992. End-to-End Scheduling to Meet Deadlines in Distributed Systems.. In *ICDCS*. 452–459.

[13] Giorgio Buttazzo, Marco Spuri, and Fabrizio Sensini. 1995. Value vs. deadline scheduling in overload conditions. In *Proceedings 16th IEEE Real-Time Systems Symposium*. IEEE, 90–99.

[14] Giorgio C Buttazzo. 2005. Rate monotonic vs. EDF: Judgment day. *Real-Time Systems* 29, 1 (2005), 5–26.

[15] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 1–15.

[16] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu, Yifan Yang, Yichi Xu, Zhilong Zheng, Yu Guan, Kun Qian, Ying Liu, et al. 2025. SyCCL: Exploiting Symmetry for Efficient Collective Communication Scheduling. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 645–662.

[17] Li Chen, Kai Chen, Wei Bai, and Mohammad Alizadeh. 2016. Scheduling mix-flows in commodity datacenters with karuna. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 174–187.

[18] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B Gibbons. 2025. SLOs-Serve: Optimized Serving of Multi-SLO LLMs. *arXiv preprint arXiv:2504.08784* (2025).

[19] Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. 2024. KVDirect: Distributed Disaggregated LLM Inference. *arXiv preprint arXiv:2501.14743* (2024).

[20] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. *See https://vicuna. lmsys. org (accessed 14 April 2023)* 2, 3 (2023), 6.

[21] Mosharaf Chowdhury and Ion Stoica. 2015. Efficient coflow scheduling without prior knowledge. *ACM SIGCOMM Computer Communication Review* 45, 4 (2015), 393–406.

[22] Mosharaf Chowdhury, Yuan Zhong, and Ion Stoica. 2014. Efficient coflow scheduling with varys. In *Proceedings of the 2014 ACM conference on SIGCOMM*. 443–454.

[23] Databriks. [n. d.]. Introducing DBRX: A New State-of-the-Art Open LLM. https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm. ([n. d.]).

[24] DeepSeek-AI. 2024. DeepEP: An Efficient Expert-Parallel Communication Library. https://github.com/deepseek-ai/DeepEP. (2024). GitHub repository, Accessed: 2026-02-07.

[25] Vignesh Ethiraj, Ashwath David, Sidhanth Menon, and Divya Vijay. 2025. Toward Low-Latency End-to-End Voice Agents for Telecommunications Using Streaming ASR, Quantized LLMs, and Real-Time TTS. *arXiv preprint arXiv:2508.04721* (2025).

[26] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024.

{Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.

[27] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.

[28] Chi-Yao Hong, Matthew Caesar, and P Brighten Godfrey. 2012. Finishing flows quickly with preemptive scheduling. *ACM SIGCOMM Computer Communication Review* 42, 4 (2012), 127–138.

[29] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. [n. d.]. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Eighth Conference on Machine Learning and Systems*.

[30] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, et al. 2025. FlashOverlap: A Lightweight Design for Efficiently Overlapping Communication and Computation. *arXiv preprint arXiv:2504.19519* (2025).

[31] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, et al. 2024. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565* (2024).

[32] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. 2024. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181* (2024).

[33] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Taiyi Wang, Bin Cui, Ana Klimovic, and Eiko Yoneki. 2025. Thunderserve: High-performance and cost-efficient llm serving in cloud environments. *arXiv preprint arXiv:2502.09334* (2025).

[34] Heehoon Kim, Junyeol Ryu, and Jaejin Lee. 2024. Tccl: Discovering better communication paths for pcie gpu clusters. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 999–1015.

[35] Kaeun Kim, Ghazal Shams, and Kawon Kim. 2025. From Seconds to Sentiments: Differential Effects of Chatbot Response Latency on Customer Evaluations. *International Journal of Human–Computer Interaction* (2025), 1–17.

[36] kimi. 2025. mooncake transfer engine. https://github.com/kvcache-ai/Mooncake/tree/main/mooncake-transfer-engine. (2025).

[37] Langfuse. 2025. How to configure retries and timeouts when fetching prompts? https://langfuse.com/faq/all/error-handling-and-timeouts. (2025).

[38] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).

[39] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 945–959.

[40] Jialong Li, Shreyansh Tripathi, Lakshay Rastogi, Yiming Lei, Rui Pan, and Yiting Xia. 2025. Optimizing Mixture-of-Experts Inference Time via Model Deployment and Communication Scheduling. *IEEE Transactions on Networking* 34 (2025), 2478–2497.

[41] Wenxin Li, Xin He, Yuan Liu, Keqiu Li, Kai Chen, Zhao Ge, Zewei Guan, Heng Qi, Song Zhang, and Guyue Liu. 2024. Flow scheduling with imprecise knowledge. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 95–111.

[42] Weiqing Li, Guochao Jiang, Xiangyong Ding, Zhangcheng Tao, Chuzhan Hao, Chenfeng Xu, Yuewei Zhang, and Hao Wang. 2025. FlowKV: A Disaggregated Inference Framework with Low-Latency KV Cache Transfer and Load-Aware Scheduling. *arXiv preprint arXiv:2504.03775* (2025).

[43] Ziyang Li, Wei Bai, Kai Chen, Dongsu Han, Yiming Zhang, Dongsheng Li, and Hongfang Yu. 2017. Rate-aware flow scheduling for commodity data center networks. In *IEEE INFOCOM 2017-IEEE Conference on Computer Communications*. IEEE, 1–9.

[44] Xudong Liao, Yijun Sun, Han Tian, Xinchen Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, et al. 2025. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 554–574.

[45] LiteLLM. 2025. Stream-timeout. https://docs.litellm.ai/docs/proxy/timeout. (2025).

[46] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[47] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).

[48] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).

[49] Kaiyuan Liu, Xiaobo Zhou, and Li Li. 2025. m2LLM: A Multi-Dimensional Optimization Framework for LLM Inference on Mobile Devices. *IEEE Transactions on Parallel and Distributed Systems* (2025).

[50] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. 2024. Rethinking machine learning collective communication as a multi-commodity flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 16–37.

[51] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.

[52] Medium. 2025. Robust LLM API Strategies. https://ai.gopubby.com/robust-llm-api-strategies-retries-fallbacks-in-python-caf9efa96908. (2025).

[53] Medium. 2025. Stream API and timeouts. https://diverger.medium.com/speeding-up-your-openai-llm-applications-f0e011f2f0d6. (2025).

[54] Meta. 2025. llama. https://huggingface.co/meta-llama. (2025).

[55] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.

[56] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.

[57] notdiamond.ai. 2025. fallbacks-and-timeout. https://docs.notdiamond.ai/docs/fallbacks-and-timeouts. (2025).

[58] NVIDIA. 2024. NVIDIA GB200 NVL72: A Revolutionary Liquid-Cooled Rack Scale Solution. https://www.nvidia.com/en-us/data-

center/gb200-nvl72/. (2024). Accessed: 2026-02-07.
[59] NVIDIA. 2025. GeForce RTX 3090 Family. https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3090-3090ti/. (2025).
[60] NVIDIA. 2025. NVIDIA Collective Communication Library (NCCL). https://github.com/NVIDIA/nccl. (2025).
[61] NVIDIA. 2025. NVIDIA Mellanox ConnectX-5. https://www.nvidia.cn/networking/ethernet/connectx-5/. (2025).
[62] NVIDIA. 2026. Dynamo: A Datacenter Scale Distributed Inference Serving Framework. https://github.com/ai-dynamo/dynamo. (2026). Accessed: 2026-02-06.
[63] NVIDIA and Dynamo Authors. 2025. Disaggregated Serving Design Document. https://github.com/ai-dynamo/dynamo/blob/main/docs/design_docs/disagg_serving.md. (2025). Accessed: 2026-02-06.
[64] NVIDIA Corporation. 2024. *NVSHMEM Documentation: Using the NVSHMEM InfiniBand GPUDirect Async Transport.* NVIDIA Corporation. https://docs.nvidia.com/nvshmem/api/using.html#using-the-nvshmem-infiniband-gpudirect-async-transport Release 3.0.6, Accessed: 2026-02-07.
[65] OpenAI. 2025. ChatGPT. https://openai.com/index/chatgpt/. (2025).
[66] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
[67] Archit Patke, Dhemath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. Queue management for slo-oriented large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 18–35.
[68] Sreeram Potluri, Pak Markthub, and Manjunath Gorentla Venkata. 2024. Enhancing Application Portability and Compatibility Across New Platforms Using NVIDIA Magnum IO NVSHMEM 3.0. https://developer.nvidia.com/blog/enhancing-application-portability-and-compatibility-across-new-platforms-using-nvidia-magnum-io-nvshmem-3-0/. (May 2024). Accessed: 2026-02-07.
[69] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.
[70] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079* (2024).
[71] Qwen Team. [n. d.]. *Qwen3-Coder-Next Technical Report.* Technical Report. https://github.com/QwenLM/Qwen3-Coder/blob/main/qwen3_coder_next_tech_report.pdf Accessed: 2026-02-03.
[72] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. {CASSINI}:{Network-Aware} job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1403–1420.
[73] Bhala Ranganathan, Mickey Zhang, and Kai Wu. 2025. Enhancing reliability in AI inference services: An empirical study on real production incidents. *arXiv preprint arXiv:2511.07424* (2025).
[74] SGLang Team. 2025. P/D Disaggregation Optimization. https://docs.sglang.ai/advanced_features/pd_disaggregation.html. (2025). Accessed: 2026-02-06.
[75] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. {TACCL}: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 593–612.
[76] Haiying Shen and Tanmoy Sen. 2025. AccelGen: Heterogeneous SLO-Guaranteed High-Throughput LLM Inference Serving for Diverse Applications. *arXiv preprint arXiv:2503.13737* (2025).
[77] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Jingliang Ren, et al. 2025. Collective Communication for 100k+ GPUs. *arXiv preprint arXiv:2510.20171* (2025).
[78] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
[79] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. D\'ej\avu: Kv-cache streaming for fast, fault-tolerant generative llm serving. *arXiv preprint arXiv:2403.01876* (2024).
[80] Hengky Susanto, Hao Jin, and Kai Chen. 2016. Stream: Decentralized opportunistic inter-coflow scheduling for datacenter networks. In *2016 IEEE 24th International Conference on Network Protocols (ICNP)*. IEEE, 1–10.
[81] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314* (2023).
[82] TL-System. 2024. FlowSim. https://github.com/TL-System/FlowSim. (2024). Accessed: 2024-05-20.
[83] Balajee Vamanan, Jahangir Hasan, and TN Vijaykumar. 2012. Deadline-aware datacenter tcp (d2tcp). *ACM SIGCOMM Computer Communication Review* 42, 4 (2012), 115–126.
[84] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. 2025. Aqua: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 48–62.
[85] vLLM Team. 2025. Prefill/Decode Disaggregation in vLLM. https://docs.vllm.ai/en/stable/features/disagg_prefill.html. (2025). Accessed: 2026-02-06.
[86] Xinchen Wan, Xinyu Yang, Kaiqiang Xu, Xudong Liao, Yilun Jin, Yijun Sun, Zhenghang Ren, Han Tian, and Kai Chen. 2025. Coflow Scheduling for LLM Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 1232–1234.
[87] Zhibin Wang, Shipeng Li, Yuhang Zhou, Xue Li, Rong Gu, Nguyen Cam-Tu, Chen Tian, and Sheng Zhong. 2024. Revisiting slo and goodput metrics in llm serving. *arXiv preprint arXiv:2410.14257* (2024).
[88] Christo Wilson, Hitesh Ballani, Thomas Karagiannis, and Ant Rowtron. 2011. Better never than late: Meeting deadlines in datacenter networks. *ACM SIGCOMM Computer Communication Review* 41, 4 (2011), 50–61.
[89] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 640–654.
[90] Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilin Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, et al. 2026. DualPath: Breaking the Storage Bandwidth Bottleneck in Agentic LLM Inference. *arXiv preprint arXiv:2602.21548* (2026).
[91] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. {AutoCCL}: Automated Collective Communication Tuning for Accelerating Distributed and Parallel {DNN} Training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 667–683.

[92] Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. 2024. Context parallelism for scalable million-token inference. *arXiv preprint arXiv:2411.01783* (2024).

[93] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.

[94] Dongha Yoon, Younghoon Min, Hoshik Kim, Sam H Noh, and Jongryool Kim. 2025. TraCT: Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale. *arXiv preprint arXiv:2512.18194* (2025).

[95] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan Suchow, Zhenyu Cui, Rong Liu, et al. 2024. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems* 37 (2024), 137010–137045.

[96] Hong Zhang, Kai Chen, Wei Bai, Dongsu Han, Chen Tian, Hao Wang, Haibing Guan, and Ming Zhang. 2015. Guaranteeing deadlines for inter-datacenter transfers. In *Proceedings of the Tenth European Conference on Computer Systems*. 1–14.

[97] Hong Zhang, Li Chen, Bairen Yi, Kai Chen, Mosharaf Chowdhury, and Yanhui Geng. 2016. CODA: Toward automatically identifying and scheduling coflows in the dark. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 160–173.

[98] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, et al. 2025. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. *arXiv preprint arXiv:2502.19811* (2025).

[99] Wei Zhang, Zhiyu Wu, Yi Mu, Banruo Liu, Myungjin Lee, and Fan Lai. 2025. Tempo: Application-aware LLM Serving with Mixed SLO Requirements. In *arXiv preprint arXiv:2504.20068*.

[100] Yangming Zhao, Kai Chen, Wei Bai, Minlan Yu, Chen Tian, Yanhui Geng, Yiming Zhang, Dan Li, and Sheng Wang. 2015. Rapier: Integrating routing and scheduling for coflow-aware data center networks. In *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 424–432.

[101] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

[102] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.

[103] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, et al. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. *arXiv preprint arXiv:2504.19442* (2025).

[104] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, et al. 2025. Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives. *arXiv preprint arXiv:2503.20313* (2025).

[105] Li Zhong and Zilong Wang. 2024. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 21841–21849.

[106] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

[107] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion control for large-scale RDMA deployments. *ACM SIGCOMM Computer Communication Review* 45, 4 (2015), 523–536.

[108] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, Shirui Lu, et al. 2025. Serving Large Language Models on Huawei CloudMatrix384. *arXiv preprint arXiv:2506.12708* (2025).

A PROOF FOR THEOREM 1

In this section, we provide the formal proof for the optimality of the *Smallest-RLI-First* policy stated in Theorem 1.

A.1 Problem Setup and Definitions

We model the prefill process as a sequence of alternating communication and computation stages across layers $\ell = 1, \ldots, N$.Let $L_{\text{curr}}(t)$ denote the layer index being computed or waiting to be computed at time t.For any communication flow f, let $L_{\text{target}}(f)$ be the layer where the data is consumed. The Relative Layer Index is defined as $\text{RLI}(f, t) = L_{\text{target}}(f) - L_{\text{curr}}(t)$.We adopt the assumptions stated in the theorem:

(i) **Ideal Computation:** Computation for layer ℓ starts immediately once its dependencies (flows with $L_{\text{target}} = \ell$) are met and runs for duration C_ℓ without interruption.

(ii) **Fluid Model:** Communication flows are infinitely divisible, allowing preemption without overhead.

(iii) **Dedicated Bandwidth:** We focus on a single link constraint (e.g., ingress or egress bottleneck).

A.2 Proof of Optimality

Objective. Minimizing the prefill makespan is equivalent to minimizing the completion time of the final computation layer. Since the total volume of computation is fixed, this is equivalent to minimizing the total duration of *Execution Stalls* (intervals where the GPU is idle waiting for data).**Interval Classification.** We partition the timeline $[0, T]$ into two types of intervals based on the state of the system:

- **Stall Intervals ($\mathcal{I}_{\text{stall}}$):** Time periods where the GPU is idle because a flow f with $\text{RLI}(f, t) = 0$ (i.e., targeting the current layer) is pending.
- **Overlap Intervals ($\mathcal{I}_{\text{overlap}}$):** Time periods where the GPU is actively performing computation. During these intervals, the link is available to transmit flows with $\text{RLI}(f, t) > 0$ (i.e., future layers).

The Exchange Argument. Suppose there exists an optimal schedule $\mathcal{S}^*$ that minimizes the makespan but violates the Smallest-RLI-First rule. This implies that at some time $t \in \mathcal{I}_{\text{stall}}$, the schedule assigns bandwidth to a flow y with a larger RLI while a flow x with a smaller RLI is pending.Specifically, since t is a stall interval, there must be some pending flow x with $\text{RLI}(x) = 0$ (blocking the current layer). The violation implies $\mathcal{S}^*$ serves flow y where $\text{RLI}(y) > \text{RLI}(x) = 0$ during $[t, t + \delta]$.We construct a new schedule $\mathcal{S}'$ by applying an exchange:

(1) **Swap:** In $\mathcal{S}'$, we assign the interval $[t, t + \delta]$ to flow x instead of y.
(2) **Effect on Stall:** By serving x earlier, the dependency for the current layer L_{curr} is satisfied δ time units earlier (assuming x was the bottleneck). This strictly reduces the duration of the current $\mathcal{I}_{\text{stall}}$ and advances the start of the next computation phase (and thus the next $\mathcal{I}_{\text{overlap}}$).
(3) **Feasibility of y:** Flow y is displaced from t. However, since $\text{RLI}(y) > 0$, flow y is not required until a future layer. The completion of x triggers the start of computation $C_{L_{\text{curr}}}$, creating a new Overlap Interval. We can safely move the transmission of y into this newly created Overlap Interval. Since y's deadline is strictly later than x's, this deferral does not violate any dependencies.

Conclusion. The constructed schedule $\mathcal{S}'$ reduces the cumulative stall duration by transforming a portion of $\mathcal{I}_{\text{stall}}$ into $\mathcal{I}_{\text{overlap}}$. Repeating this exchange argument for all violations proves that the schedule strictly prioritizing the smallest RLI (serving RLI = 0 flows immediately to minimize Stalls, and using Overlap intervals for RLI > 0 flows) yields the minimum makespan. □

B FULL ALGORITHM FOR ROBUST INTER-REQUEST SCHEDULING

The overall workflow of the scheduling algorithm is illustrated in Algorithm 1. In this section, we elaborate on the scheduling logic, providing the comprehensive mathematical formulation of priority assignment and further details on the practical implementation of latency estimation and enforcement.

This section details the robust scheduling algorithm designed to handle imprecise laxity and overload. The workflow operates in three phases triggered by batch arrival and departure events.

Step 1: Sorting via Robust Effective Deadline. To mitigate the "Piggyback Effect," the scheduler employs the Robust Effective Deadline (*RED*) metric defined in §4.4.2. The prioritization process executes in two stages. First, for each batch $\mathcal{B}$ with n requests, the system sorts the requests internally by their deadlines $d_1 \le d_2 \le \cdots \le d_n$. It then performs a linear scan to identify the partition point $k^* = \arg\max_{1 \le k < n}(d_{k+1} - d_k)$ that yields the maximal inter-request gap. This calculation dynamically separates the outliers (Tight Set) from the majority (Loose Set). Based on this partition, the scheduler computes the *RED* score using the derived sub-batch deadlines and the outlier ratio f. Finally, all active batches are organized into a global priority queue sorted by ascending *RED* values. This ensures that the dispatch order effectively filters out transient urgency spikes from isolated outliers while preserving the priority of genuinely urgent workloads.

Algorithm 1 Inter-request Scheduling

procedure InterScheduling($\mathcal{B}$, M, B)
▷ **input** $\mathcal{B}, M$: Batches and Traffic Matrices
▷ **input** B: Global total drop budget
▷ **output** $\sigma, \mathcal{H}$
  $S(\cdot) \leftarrow 0$ ▷ Interference from high-priority batches
  $\mathcal{P} \leftarrow \emptyset$; $\mathcal{H} \leftarrow \emptyset$
  ▷ Step 1: Sorting via Effective Deadline
  $\sigma \leftarrow$ SortByRED($\mathcal{B}$)
  Precompute load vectors L from M
  for $k \leftarrow 1$ **to** $|\sigma|$ **do**
    Let $i \leftarrow \sigma[k]$
    $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{B}_i$ ▷ Add to candidate pool
    $\widehat{F}_i \leftarrow$ EstFinishTime(S, L_i)
  ▷ Step 2: Feasibility Check
    while $\widehat{F}_i > D_i^{Lo}$ **and** $|\mathcal{H}| < B$ **and** $\mathcal{P} \neq \emptyset$ **do**
      $u^\star \leftarrow \arg\max_u (S(u) + L_i(u))$ ▷ Bottleneck
  ▷ Step 3: selective pruning
      $r^\star \leftarrow \arg\max_{r \in \mathcal{P}}$ Load of r on $u^\star$
      $\mathcal{H} \leftarrow \mathcal{H} \cup \{r^\star\}$; $\mathcal{P} \leftarrow \mathcal{P} \setminus \{r^\star\}$
      if $r^\star \in \mathcal{B}_i$ **then** ▷ Drop request in current batch
        $L_i(\cdot) \leftarrow L_i(\cdot) - \ell_{r^\star}(\cdot)$
      else ▷ Drop request in higher priority
        $S(\cdot) \leftarrow S(\cdot) - \ell_{r^\star}(\cdot)$
      $\widehat{F}_i \leftarrow$ EstFinishTime(S, L_i)
    $S(\cdot) \leftarrow S(\cdot) + L_i(\cdot)$ ▷ Update
  return $\sigma, \mathcal{H}$

Step 2: Worst-case Feasibility Check. Following prioritization, we perform an admission control check to prevent the "Black Hole Effect." We estimate the completion time $\widehat{T}_i$ by strictly accumulating delays under a worst-case assumption (no overlap between batches). Computation latency is treated as deterministic, derived from the static computation graph of the Transformer model and offline profiling on the target hardware [5, 70, 106]. Communication latency is estimated by analyzing the batch's traffic matrix to identify the bottleneck port p^* in the network fabric; the delay is calculated as the cumulative load on p^* divided by its bandwidth. For dynamic architectures like Mixture-of-Experts (MoE) where token routing is runtime-dependent, we rely

on historical routing statistics. Prior studies [40] indicate that despite high noise in individual traffic matrices, the end-to-end latency prediction error typically remains within 20%. This level of accuracy is sufficient for our scheduling granularity, allowing us to reliably enforce feasibility against the target deadline ($D_{\min}^{Lo}$).

Step 3: Selective Pruning and Soft Enforcement. If a batch fails the feasibility check (i.e., $\widehat{T}_i > D_{\min}^{Lo}$), the system triggers a surgical pruning mechanism. We identify the specific requests within the batch that contribute the maximum load to the bottleneck port p^* and iteratively remove them from the feasible set until the predicted latency meets the deadline. Crucially, this pruning employs a *soft enforcement* strategy. Instead of immediately discarding the pruned requests, the scheduler demotes them to a "Scavenger" priority class within a dual-queue system. The Main Queue is serviced with strict priority, while the Scavenger Queue is processed opportunistically only when the network is detected to be idle or when actual runtime latencies are lower than the pessimistic estimates. This hybrid approach ensures system stability under worst-case predictions while maximizing throughput by reclaiming resources when congestion does not materialize.

Summary. It is important to emphasize the design hierarchy within this workflow. The RED metric acts as the primary tie-breaker, governing the global dispatch order to maximize service-level objective (SLO) compliance under normal to moderate load. The feasibility check and selective pruning operate strictly as complementary safeguards, activated only during pathological overload to prevent resource exhaustion. This separation of concerns ensures that the scheduler remains stable and predictable—relying on robust prioritization for the vast majority of decisions—while retaining the capability to degrade gracefully only when extreme contention makes it unavoidable.